AgentProf: Semantic Profiling for AI Agents

Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve quality, safety, and cost efficiency, developers need to determine which task categories consume the most budget, where failures concentrate, and which workflows trigger unsafe effects. To answer similar questions in traditional systems software, profiling attributes resource consumption to responsible entities, such as code paths, by aggregation, rather than single-execution debugging or tracing. Yet existing agent observability tools are mainly for debugging and tracing instead of profiling. Agent behavior differs fundamentally from code execution. The responsible entities are semantic categories such as task intent and workflow phase, not code paths, and agent trajectories lack the stable identifiers and structural hierarchies that make aggregation possible. We propose a *semantic operation stack model* that adapts profiling to agent trajectories. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack with query-time field projections that fold identical frame sequences. AgentProf implements this model as a profiler with pluggable algorithms for *intent attribution* and *stack construction*, compiling local agent trajectories into pprof-compatible profiles. On real trajectories and public datasets, our evaluation shows that semantic profiling effectively attributes cost and locates problems across agent trajectories.

1 Introduction

AI agents [1, 31, 33] orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours and days, with a large number of interactions each, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Answering these questions requires analysis and evaluation across many trajectories [22, 23], a task becoming costly for manual inspection or per-trajectory LLM evaluation [36] at scale. In traditional systems software, profiling answers these questions by attributing resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing.

Yet existing agent tools support debugging and tracing but not profiling (§2). Some [2, 14, 15, 25] organize events into per-execution span trees, effective for diagnosing a single run but unable to aggregate by semantic category. Others [7, 13] cluster inputs or extract structured signals, but characterize *input distributions* ("30% of users ask code questions") rather than *resource attribution* ("review tasks consume 40% of the token budget"), because tags at the request level do not propagate to downstream tool calls and system effects.

Adapting profiling to agent trajectories is challenging (§2). In traditional software, the responsible entities are code paths. Profiling is straightforward because function names are stable and runtime call nesting provides the aggregation hierarchy (§2). Agent behavior differs fundamentally. To answer the questions above, the responsible entities need to be semantic categories such as task intent and tool operation, not code paths. Agent trajectories also lack the two properties that make aggregation possible. Prompts are natural language, and two prompts expressing the same intent share no common string. Framework-level span names (`LLMCall`, `ToolInvoke`) describe mechanisms, not semantic intent. Agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames.

Despite these differences, the fundamental profiling methodology (define units, assign stable tags, construct aggregation hierarchies, fold identical stacks) transfers to agent trajectories. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* are ordered lists of fields chosen at query time. Each operation is projected to a frame sequence, and operations with identical sequences merge, producing the folded-stack representation that flame graphs visualize (Figure 1). Different field selections answer different questions (by task category, by execution phase, by session, or by action type) from the same data.

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Figure 2). Two pluggable algorithm frameworks enrich the model. *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This gives agents the stable tags that traditional profiling gets for free from function names. *Stack construction* builds aggregation hierarchies from available fields, either from a user-specified field list or by automatically detecting boundaries between distinct work phases. On 325 real Codex and Claude trajectories (183,714 system observations), semantic profiling separates over 90% of system-effect cost that per-session views cannot attribute to task categories. On four public datasets (34,539 operations) where the correct annotations are known but hidden from the profiler, ranked groups locate real problems at 9.4% inspection work, with 45% fewer groups than per-session grouping. Derived tags agree with ground truth on 7 of 9 held-out datasets.

This paper makes three contributions:

(1) **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (§2–§3).
(2) **System: AgentProf.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output (§4).

Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. *Top (tokens):* width represents token count. *Middle (time):* width represents wall-clock duration. *Bottom (files):* width represents file-operation count. All three are produced from the same operations by changing only the stack fields and weight function (Table 1).

(3) **Evaluation.** We evaluate profiler output against hidden ground-truth trajectory annotations, analogous to injecting known bottlenecks in CPU profiler evaluation, on 325 real trajectories and 4 public datasets (§5).

2 Background and Motivation

2.1 LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers system effects at the lower layer: process spawns, file reads and writes, and network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

2.2 System Profiling

Profiling attributes resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing. In traditional software, the profiling pipeline works as follows: a profiler periodically samples execution state, attaches each sample to the current call stack, and merges samples with identical stacks. Flame graphs [12] and pprof [11] call graphs visualize the result, where width or node size represents aggregate cost. Perfetto [10] generalizes this with SQL-based analysis and derived events. These tools support flexible aggregation, but they all assume stable function names and a runtime call stack. Pprof's `tagroot/tagleaf` options can add label-derived pseudo-frames, but only on top of an existing execution stack that agent trajectories do not have.

2.3 Challenges for Agent Profiling

Existing tools [2, 14, 15, 25] record agent events as per-execution span trees, effective for debugging a single run. AgentSight [37] bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting traces still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing tools do not capture these categories:

sessions are boundaries, not categories (a single session may mix review, debugging, and refactoring phases), and span names [3, 25] describe mechanisms (`LLMCall`, `ToolInvoke`), not user intent (`review`, `debug`). The second challenge is that agent trajectories lack the two properties that make traditional aggregation possible: prompts are natural language without stable identifiers, and agent events have no runtime call-stack hierarchy. As a result, a common system effect like running a test suite may appear thousands of times across trajectories, but without semantic tags no tool can attribute these executions to the task categories that triggered them. We quantify this in RQ1 (§5).

2.4 Design Requirements for Agent Profiling

These observations yield three design requirements:

- **R1: Uniform record type.** Agent activities mix prompts, LLM calls, tool invocations, GUI actions, process events, and benchmark annotations. The profiler needs a single record type that can represent all of them without type-specific objects.
- **R2: Query-time aggregation.** The same data must support multiple aggregation shapes (by task, phase, action, session, or dataset) without rebuilding the input. Sessions and spans must be fields, not fixed hierarchy nodes.
- **R3: Stable tags for folding.** Free-form prompts must be mapped to short, stable, repeatable tags before folding. This *intent attribution* step has no analogue in CPU profiling, where function names are already stable identifiers.

3 Design

The three requirements (a uniform record type, query-time aggregation, and stable tags for folding) drive the design. The semantic operation stack model has two components, operations (§3.1) and operation stacks (§3.1), satisfying R1 and R2. Two pluggable algorithm frameworks enrich the model. Intent attribution (§3.2) satisfies R3 by deriving stable tags, and stack construction (§3.1) builds aggregation hierarchies from available fields. The profiling pipeline has five stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) filter by a user-specified predicate, (4) project each operation onto a stack frame sequence chosen at query time, and (5) fold operations with identical stacks, summing their weights.

3.1 Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in §2, a profiler should not distinguish between them in the data representation. AgentProf therefore represents every activity as a single abstraction, the `operation`, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, occurrence count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operation records that differ only in which fields are populated.

An `operation stack` is an ordered list of fields chosen at query time. Given a stack specification $[f_1, f_2, \ldots, f_k]$, each operation o is projected to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$. Operations with identical frame sequences are merged and their weights are

Table 1: Built-in views. Each shares the same operation set but selects different activities, stack structures, and weights.

View	φ (select)	w (width)	Question
`tokens`	LLM calls	token count	Budget distribution?
`time`	All timed ops	duration (s)	Time distribution?
`files`	File-effect ops	event count	Repo coverage?
`network`	Net-effect ops	event count	External contacts?

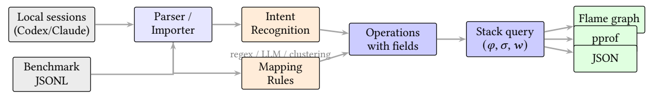

Figure 2: System architecture. Local agent histories enter through `agent-session` parsing. Public datasets enter as operation JSONL. Both paths produce operations. The stack query and output are identical.

summed. Unlike CPU profiling, where the call stack is determined by execution, the stack is determined by the query. The same operations can be folded as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory using the dataset's own annotations, and changing the fields changes the aggregation without changing the data. Sessions and spans are optional operation fields, not structural skeleton nodes, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude it). A view is a triple (φ, σ, w): a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Four built-in defaults (Table 1) answer common questions.

3.2 Algorithms

Intent attribution. Intent attribution maps free-form prompts to short, stable tags, giving agent trajectories the stable identifiers that traditional profiling gets for free from function names (§2). The framework supports three pluggable backends. Regex rules are the production default: rules have the form `KIND:TAG=REGEX`, are evaluated in order (first match wins), and the developer iterates with AgentProf until the unmatched rate drops below 5%, typically in 5–10 rounds. A local LLM tagger (e.g., a quantized 3B-parameter model via llama.cpp) generates a single-word tag per prompt with grammar-constrained decoding (forcing output to match a predefined token grammar) and caching. Unsupervised clustering via TF-IDF and K-Means discovers natural prompt groups without predefined rules, feeding rule authoring. For structured trajectories where action types and quality annotations are explicit fields, mapping rules derive new fields from existing ones (e.g., `phase:navigate=type:(click|scroll|key)`). All backends are pluggable, and a profiling configuration records the complete analysis setup for deterministic replay.

Stack construction. Stack construction builds aggregation hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When no field list is supplied, AgentProf can induce operation stacks automatically by scoring adjacent operations for topic changes, field changes, and group consistency, then cutting at boundaries where the evidence is strongest. The induced stacks have variable depth, and a configurable depth cap controls the granularity. This mode produces coarser groups than hand-specified stacks but requires no domain knowledge, making it useful for initial exploration (§5).

4 Implementation

AgentProf is an offline, post-hoc profiler implemented as a Rust CLI (Figure 2). It reads recorded trajectories, not live events. The `agent-session` crate reads Codex and Claude Code local JSONL history files and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Each prompt opens a span. LLM calls, tool calls, and system effects within that span inherit its semantic tag, connecting intent to system behavior without runtime instrumentation. Public benchmark datasets enter as operation JSONL with mapping rules deriving fields before stack construction. Standard Chrome Trace Event format can also be imported, with non-AgentProf protocol fields preserved as operation fields.

Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries. Operations can also be exported as standard Chrome Trace Event JSON and re-imported with byte-identical folded output, enabling interoperability with trace viewers such as Perfetto. All outputs are deterministic given the same input and profiling configuration.

AgentProf is the offline profiling component of the AgentSight [37] observability framework. AgentSight provides real-time eBPF-based capture of SSL traffic, process events, and file/network effects. AgentProf aggregates the recorded trajectories into semantic profiles, so the two are complementary: AgentSight answers "what happened during this run" (debugging), and AgentProf answers "where did the budget go across runs" (profiling).

5 Evaluation

We use two classes of data. **Real agent trajectories**: 325 readable local trajectories (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project, containing 183,714 system observations. **Public annotated trajectories**: 15 families of real agent or human executions, totaling 47,590 operations through mapping rules. These cover web agents (WebLINX [21], Mind2Web [8], AgentTrek [32], WebShop [34]), API agents (API-Bank [16], ToolBench [26], tau-bench [35]), coding agents (SWE-agent [33]), and mobile/GUI agents (AndroidCtrl [17], GUI-Odyssey [20], Scale-CUA [19]). Four families with ground-truth quality annotations (AgentRewardBench [22], SATraj-OS [6], AgentNet [28], OSWorld-Human [30]) provide hidden annotations for RQ2. We measure semantic separation via the mixed-weight percentage defined in RQ1. For localization quality (RQ2), we use average precision (AP, the area under the precision-recall curve when groups are ranked by suspected problem density), recall at various inspection budgets, and work-to-first-positive (the fraction of groups inspected before finding one positive). For tag agreement (RQ3), we use V-measure (a clustering-agreement score ranging from 0 to 1) and boundary F1.

5.1 RQ1: Does Semantic Profiling Improve Resource Attribution?

The strongest argument against semantic profiling is that traditional tools "can already see the same information." To test this, we run a semantic-axis ablation on 325 real Codex/Claude trajectories (183,714 system observations). We progressively add tag axes (no tags, session only, prompt only, both) and measure the mixed-weight percentage, the fraction of system-effect cost in groups that mix observations from multiple task categories. A lower mixed-weight percentage means the profiler correctly separates effects by task category. The residual percentage measures weight in groups where the majority category accounts for less than 90% of the group, capturing partial mixing.

Figure 3 shows the result. Without semantic tags, nearly all system-effect cost falls into mixed groups: a flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. The prompt tag is the decisive axis: it drops both mixed weight and residual by more than half and nearly doubles the number of unique stacks, because it separates observations that share the same system call but originate from different task intents. The session tag alone barely helps because trajectories typically contain multiple intent phases.

A tag-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. Semantic-tag quality is validated in RQ3.

Beyond tag separation, we test whether choosing different field selections for the operation stack produces views that flat grouping cannot. We take the same 13,265 operations from the four families with ground-truth annotations and fold them with five different field selections, counting how many distinct groups each produces. The same operations fold into 9 dataset groups, 57 phase groups, 226 semantic groups, 455 action groups, or 3,757 per-session groups by changing only the stack fields. A flat GROUP BY on a single tag cannot produce these views because different stack depths answer different questions (budget by task vs. duplication by action vs. per-session detail). The same model also covers all 15 public trajectory families without type-specific objects.

Different views also produce different rankings. The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens, showing that the choice of weight function changes the analysis conclusions.

Finally, we test whether the profiler can produce useful groups without any user-specified fields. We run the automatic stack-induction mode (§3.1) on the same six tasks used in RQ2 and compare the resulting group count and localization quality against flat summaries and per-session grouping. Induced stacks produce a median of 12 groups (versus 285 for per-session and 1 for flat), with 4/6 tasks generating variable-depth stacks. The induced stacks achieve lower AP than hand-configured stacks (median 0.276 vs 0.312) because they use only visible fields, but they reduce inspection work to 65.3% versus flat summaries without requiring any user configuration.

Figure 1 shows the resulting flame graphs for the `tokens` and `files` views, illustrating that changing only the stack fields and

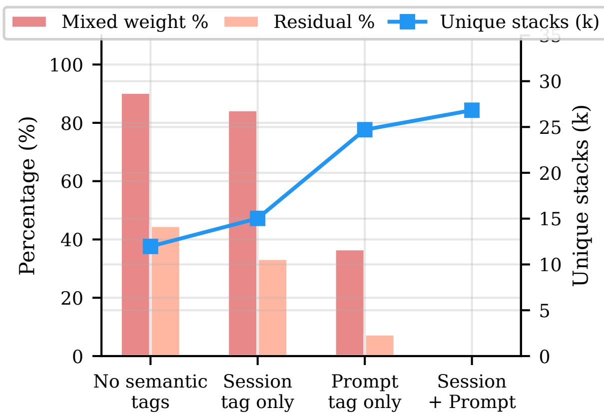

Figure 3: RQ1: Semantic-axis ablation on 183,714 system observations from 325 real trajectories. Bars show mixed weight and residual percentages (left axis); the line shows unique stacks (right axis). Adding prompt tags reduces mixed weight from 90.4% to 36.7% while increasing unique stacks from 12k to 25k. Session tags alone barely help (84.4%).

weight function produces different aggregates from the same operations.

5.2 RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 tests whether the profiler's top-ranked groups actually contain real failures, safety violations, or wasted effort. We use public benchmark datasets that provide ground-truth quality annotations (failure, safety violation, redundant step, human-task boundary), hide these annotations from the profiler, and check whether the top-ranked groups recover them, analogous to injecting known bottlenecks in a CPU profiler evaluation.

The benchmark uses six tasks across four datasets (AgentRewardBench looping and side-effect, SATraj-OS unsafe operation, AgentNet incorrect and redundant step, and OSWorld-Human group start), totaling 34,539 operations and 3,699 positives. We compare six grouping strategies (flat summary, per-session grouping, native hierarchy, raw-action grouping, operation-stack profiling, and an oracle that uses the hidden annotations) and rank groups by the fraction of hidden positives they contain.

Table 2 summarizes the results. Flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Per-session grouping finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack profiling inspects 9.4% of the work in the top five groups, recovers 39.0% of positives within 30% of the inspection budget, and reduces groups to 157.5.

Figure 4 visualizes this tradeoff: operation-stack profiling sits in the low-work, moderate-recall region. The comparison also serves as evidence for the operation/operation-stack model (§3). Per-session grouping and native hierarchy are alternative abstractions that fix the hierarchy at session or dataset boundaries. Operation stacks subsume both as special cases (include `session` in

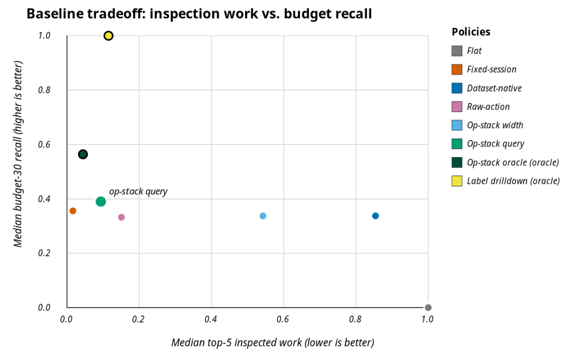

Figure 4: RQ2: Inspection work vs. recall at 30% inspection budget. Operation-stack profiling (tagged) achieves low work (0.094) with moderate recall (0.39), while flat requires full inspection and per-session fragments into 285 groups. Oracle policies (large dots) are upper bounds.

the stack to recover per-session; include `dataset` to recover the native hierarchy). On localization, operation stacks improve top-five recall over per-session on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Per-session grouping retains lower work-to-first-positive on 4/6 tasks, confirming that profilers and debuggers are complementary.

Statistical validation supports these comparisons. Bootstrap (10,000 resamples) and tag-permutation (2,000 trials) confirm that the AP differences are stable and exceed the 95% confidence threshold on all six tasks.

We also test whether stack depth matters by sweeping the induced-stack depth cap from 1 to 5 on the same six tasks and measuring AP and inspection work at each depth. AP peaks at depth 3 (median 0.287) while inspection work is lowest at depth 5 (median 0.473). Different tasks achieve their best AP at different depths (2, 3, 4, and 5), confirming that depth should be tuned per objective rather than fixed globally.

The profiler's configuration is also actionable. Modifying stack fields, mapping rules, or ranking criteria and re-running on the same operations improves AP on 5/6 tasks (median AP improvement 0.038), confirming that the profiler exposes useful tuning knobs rather than requiring a fixed configuration.

5.3 RQ3: How Accurate Are the Tags?

RQ1 and RQ2 assume that semantic tags (the tags derived by intent attribution or mapping rules) are useful. RQ3 tests whether these semantic tags are accurate by comparing them against ground-truth annotations from the benchmark datasets. For real trajectories with free-form prompts, we measure whether the intent-attribution backends run reliably at scale and produce well-formed tags. Semantic accuracy of the LLM/regex tagger on free-form prompts remains future work because none of the available datasets (67,304 rows across 6 sources) simultaneously expose free-form text and ground-truth semantic tags, making same-input accuracy evaluation infeasible with current data.

Table 2: RQ2 hidden-annotation localization (medians across six tasks). AP = average precision. R@5 = recall in top 5 groups. Work@5 = fraction of total work in top 5 groups. R@30% = recall at 30% inspection budget. WTFP = work-to-first-positive (fraction inspected before finding one positive). Hidden = 1 means the policy uses hidden annotations (oracle upper bound). "—" = group count varies by task.

Policy	Hidden	AP	R@5	F1@5	Work@5	R@30%	WTFP	Groups
Flat (all-in-one)	0	0.168	1.000	0.287	1.000	0.000	1.000	1.0
Per-session drilldown	0	0.348	0.023	0.043	0.016	0.356	0.004	285.0
Native hierarchy	0	0.357	0.867	0.282	0.854	0.338	0.058	—
Raw-action grouping	0	0.274	0.244	0.220	0.151	0.333	0.037	—
Operation-stack profiling	0	0.312	0.188	0.198	0.094	0.390	0.038	157.5
Op-stack + oracle fields	1	0.599	0.300	0.403	0.044	0.564	0.012	—
Oracle (hidden annotations)	1	1.000	0.670	0.797	0.115	1.000	0.038	—

Table 3: Leave-dataset-out mapping quality. V-measure between mapping-derived `phase` and native `action` annotations. 13,265 operations from the four annotated families across 9 held-out evaluations.

Held-out dataset	Ops	V-measure	Boundary F1
mind2web	774	1.000	1.000
webshop-expert	2,504	1.000	1.000
swe-agent	496	0.926	0.962
weblinx-chat	500	0.872	0.860
agenttrek	200	0.862	—
gui-odyssey	7,868	0.811	0.842
android-control	9	0.716	0.727
toolbench	866	0.134	0.353
api-bank	48	0.000	—

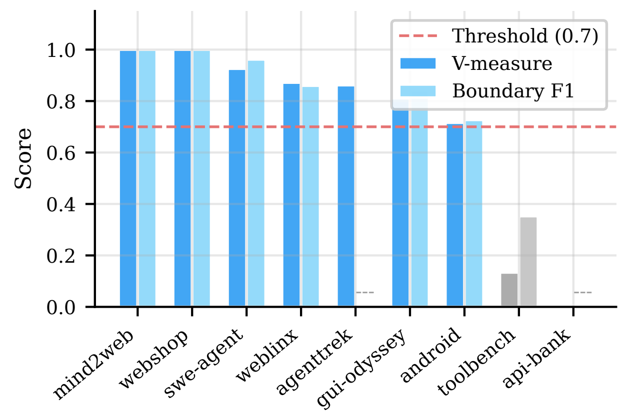

Figure 5: RQ3: Leave-dataset-out V-measure between mapping-derived `phase` and native `action` annotations. The dashed line marks 0.7. Seven of nine datasets exceed this threshold.

On 325 real trajectories, the LLM tagger processes 118,021 tag requests with zero failures, 100% syntax validity, and 70% cache hit rate (operational details in RQ4).

Public benchmark datasets provide native action annotations that serve as ground truth for mapping quality. We run a leave-dataset-out evaluation. For each of 9 datasets, we train mapping rules on the other 8 and measure V-measure between the mapping-derived `phase` field and the dataset's native `action` annotations, plus boundary F1 between mapped phase boundaries and native action boundaries.

Table 3 and Figure 5 show the results. On 7/9 datasets, V-measure exceeds 0.7, indicating strong agreement between mapping-derived tags and native annotations. The two low-scoring datasets (toolbench, api-bank) have atypical action vocabularies that the held-out mapping rules do not cover.

Supervised boundary predictors beat simple baselines on 4/5 tested datasets (e.g., F1 0.77 vs 0.71 on OSWorld-Human), confirming that mapping rules produce useful tags but are not automatic intent discovery.

5.4 RQ4: What Is the Profiling Cost?

AgentProf is an offline, post-hoc profiler, so it adds zero overhead to the agent's runtime. The profiling cost is the time and resources spent after the session ends.

We re-execute 76 tracked profiling configurations twice (152 invocations) to measure cost and verify determinism. All 76 configurations produce byte-identical output across both runs. Median profiling time is 1.6 s per configuration (p95 2.8 s, max 4.3 s), and median output size is 37.5 KB. The slowest configurations process views with over 600 unique stacks, while the 60 robustness configurations all complete within 1.6 s. The full configuration space (operation files, query predicates, ranking criteria, and stack depth) stays deterministic across all 12 tested variants.

Intent attribution adds a one-time cost for new prompts. Of 118,021 tag requests on 325 real trajectories, 70% are cache hits (zero cost), and the remaining 35,136 new calls complete at p95 latency of 31 ms per tag using a local 3B-parameter model. Grammar-constrained decoding ensures 100% syntax validity (2,700/2,700 test tags). Because tags are cached, re-profiling the same trajectories with different stack fields or predicates incurs no additional LLM calls.

6 Related Work

Agent observability tools [2–4, 9, 14, 15, 25, 29] focus on single-execution debugging. Datadog LLM Observability [7] and Laminar [13] cluster inputs or extract signals but characterize input distributions, not resource attribution. AgentRx [5], TELBench [27], AgentAtlas [23], TrajAD [18], and AgentFixer [24] perform per-trajectory fault localization. AgentProf complements all of these with category-level aggregate profiling across trajectories.

7 Conclusion

Agent observability needs profiling, not only debugging. AgentProf shows that the semantic operation stack model, combined with pluggable intent attribution and stack construction algorithms, closes the aggregation gap for agent trajectories. Scoring profiler output against hidden trajectory annotations confirms that semantic profiling separates over 90% of system-effect cost that per-session views leave mixed, localizes annotated problems at 9.4% inspection work versus flat summaries, and produces tags that agree with ground truth on 7/9 held-out datasets. Per-session grouping remains faster to a first positive on 4/6 tasks, confirming that profilers and debuggers are complementary, and AgentProf supports both by including or excluding session fields in the stack query. Multi-project evaluation and trace-ecosystem integration are the most impactful next steps.

References

[1] Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.
[2] Arize AI. 2024. Arize Phoenix. https://arize.com/docs/phoenix.
[3] Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.
[4] Krishna Chaitanya Balusu. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*. doi:10.1145/3805760.3814931
[5] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475. doi:10.48550/arXiv.2602.02475
[6] Xinquan Chen, Zhenyun Yin, Shan He, Bin Huang, Shanzhe Lei, et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230. doi:10.48550/arXiv.2605.06230
[7] Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
[8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*. doi:10.5555/3666122.3667342
[9] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285. doi:10.48550/arXiv.2411.05285
[10] Google. 2024. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.
[11] Google. 2024. pprof. https://github.com/google/pprof.
[12] Brendan Gregg. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.
[13] Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.
[14] LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.
[15] Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.
[16] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3102–3116. doi:10.18653/v1/2023.emnlp-main.187
[17] Wei Li, William E. Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679. doi:10.48550/arXiv.2406.03679
[18] Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443. doi:10.48550/arXiv.2602.06443
[19] Zhaoyang Liu, Jingjing Xie, Zichen Ding, Zehao Li, Bowen Yang, et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221. doi:10.48550/arXiv.2509.15221
[20] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botian Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. https://github.com/OpenGVLab/GUI-Odyssey
[21] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930. doi:10.48550/arXiv.2402.05930
[22] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942. doi:10.48550/arXiv.2504.08942
[23] Parsa Mazaheri and Kasra Mazaheri. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530. doi:10.48550/arXiv.2605.20530
[24] Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. In *Proceedings of the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE)*. doi:10.48550/arXiv.2603.29848
[25] OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.
[26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerber, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=dHng2O0Jjr
[27] Jiaming Wang, Ziteng Feng, Jiangtao Wu, Ruihao Li, Qianqian Xie, Yuxiang Ren, He Zhu, Xueming Han, Fanyu Meng, Junlan Feng, and Jiaheng Liu. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060. doi:10.48550/arXiv.2606.02060
[28] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.
[29] Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Manqing Dong, Mingkai Zheng, Xuefei Yin, and Yanming Zhu. 2026. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990. doi:10.48550/arXiv.2606.04990
[30] WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.
[31] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shi, Joel Tong, Kai-Wei Lu, Vedant Garg, Yitao Wang, Ziniu Dai, Fangyu Li, Yonatan Bisk, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
[32] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605. doi:10.48550/arXiv.2412.09605
[33] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. doi:10.5555/3737916.3739517
[34] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. doi:10.5555/3600270.3601778
[35] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045
[36] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*. https://arxiv.org/abs/2306.05685
[37] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the*

4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP). doi:10.1145/3766882.3767169